$0<\epsilon,\delta<1/2$,

$$\frac{1}{T}\sum_{t=1}^{T}\|p(x_t|x_1,...,x_{t-1})-p'(x_t|x_1,...,x_{t-1})\|_1 \leq \epsilon$$

with probability greater than $1-\delta$*, when* $T \geq \frac{d}{\epsilon^4}\,\mathrm{polylog}(d,\frac{1}{\epsilon},\frac{1}{\delta})$*, where the polylogarithmic term hides an absolute constant that is independent of* p*.*

This theorem says that this associated lexinvariant language model converges to modeling the true language distribution fairly efficiently—with polynomial rate and near-linear dependence on vocabulary size d. Strikingly, this holds irrespective of the properties of the language distribution p [2]. In other words, *a language model can indeed infer the operational meaning of the tokens in context based solely on the structure of the symbols*!

We give a complete proof of this theorem in Appendix A. At a high level, this convergence happens because at most timesteps t, the new observation x_t either provides new information about the permutation π, or x_t has similar likelihood under the permutations that are likely given $x_1,...,x_{t-1}$. In the simplest case, if the posterior $p(\pi\,|\,x_1,...,x_n)$ concentrates on the correct π, then we converge to the standard LM. But even if it doesn't, that means the uncertainty about π should not matter for predicting the next token. We make this precise by interpreting $p'(x_t|x_1,...,x_{t-1})$ as performing a multiplicative weights algorithm with the Hedge strategy of Freund and Schapire [8], and then relate the regret bounds to the average KL divergence between the predictions of p and p', and ultimately the average $\mathcal{L}_1$ distance between these predictions.

2.2 In-context Bayesian Deciphering

We can see the associated lexinvariant language model as implicitly learning to approximate an in-context Bayesian deciphering process, i.e. inferring a probability distribution over possible lexical permutations based on seen tokens, with the language modeling prior:

$$\begin{aligned} &p'(x_{n+1}|x_1,...,x_n) \\ =&\sum_{\pi}\underbrace{p(\pi^{-1}(x_{n+1})|\pi^{-1}(x_1),...,\pi^{-1}(x_n))}_{\text{language modeling}}\ \underbrace{\mathcal{P}(\pi|x_1,...,x_n)}_{\text{inferring lexical permutation}} \end{aligned} \tag{3}$$

As shown above, p' can be reduced to two parts, where the first part is normal language modeling and the second part is the probability distribution of lexical permutations based on seen tokens. So the lexinvariant language model is implicitly learning to model $\mathcal{P}(\pi|x_1,...,x_n)$.

We can make this approximate in-context Bayesian deciphering explicit by training a small probe to predict $\mathcal{P}(\pi|x_1,...,x_n)$ given the internal representation of the lexinvariant language model. We will show that this indeed recovers π reasonably accurately in the experiment section.

2.3 Constructing a Lexinvariant Language Model

We now consider how to construct a lexinvariant LM in practice. A typical neural language model, such as a Transformer, converts input tokens to continuous vectors using token embedding and then passes these vectors as input to the rest of the neural network. Thus, the language model p it parameterizes depends on the token embedding $E:\mathcal{V}\rightarrow\mathbb{R}^d$:

$$p(x_1,...,x_n)=T(E(x_1),...,E(x_n)) \tag{4}$$

To make a neural LM lexinvariant, we can replace the standard stable token embedding E with a randomized E and take the expectation over E. Each token $x\in\mathcal{V}$ has an independent embedding $E(x)\sim\mathcal{N}(0,\mathcal{I}_d)$, and the language model becomes

$$p(x_1,...,x_n)=\mathbb{E}[T(E(x_1),...,E(x_n))] \tag{5}$$

Since $E\overset{d}{=}E\circ\pi$, the right-hand side is the same when x_i are applied with any permutation π, i.e., for any $x_1,...,x_n$:

$$\mathbb{E}[T(E(x_1),...,E(x_n))]=\mathbb{E}[T(E(\pi(x_1)),...,E(\pi(x_n)))], \tag{6}$$

[2]The convergence rate could be better depending on the language distribution, such as on math and code, where the symbols should have clear functional meaning in context. We explore this empirically in the experiment section.

showing that the Transformer with random E is lexinvariant as in Eq. 1. Now we can train this lexinvariant LM similarly to a standard LM. Concretely, we sample a new E for each training sequence and minimize the standard language modeling loss as in a standard neural LM. Here we are stochastically optimizing a variational lower bound of the standard language modeling loss with this randomized model by taking the expectation to the outside of the loss over log likelihood. Effectively, the same token gets the same random embedding within each training sequence, but different embedding across training sequences.

In practice, we focus on training decoder-only Transformers with a next token prediction objective in this work, where the model directly models $p(x_{n+1}|x_1,...,x_n)$ instead of the joint distribution. Our definitions and analysis above still hold in general. The only modification is that the final readout matrix also needs to be replaced with the same E, so that the Transformer can predict the embedding of the next token based on the embeddding of input tokens.

3 Experiments

3.1 Setup

Architecture. For all experiments, we use decoder-only Transformer architecture with T5 relative position bias [19]. We use models with 150M parameters, with 12 layers, 8 heads, head dimension 128, and MLP dimension 4096.

Training. We use the Adafactor optimizer [22], with a cosine decay learning rate schedule [13] from 0.01 to 0.001 based on preliminary experiments. We train the models from scratch for 250K steps on all the settings, with 512 sequence length and 64 batch size. We ran all of our experiments on 8 TPU cores. Our models are implemented in JAX [5].

Datasets. For datasets, we mainly use the Pile [9], a large open-source corpus that contains text collected from 22 diverse high-quality sources. We also run experiments on two additional datasets to explore their effects on the behavior of lexinvariant models: Wiki-40B [10], which contains high quality processed Wikipedia text in 40+ languages, and GitHub (subset of the Pile), which contains code from GitHub repositories with more than 100 stars and less than 1GB files.

3.2 Convergence to Standard Language Models

We first show empirically that lexinvariant LMs can mostly recover the next token prediction performance of standard LMs after a long enough context. As already discussed in section 2.1, the lexinvariant LM will theoretically converge to a standard LM as the context becomes long enough to resolve ambiguity. Here we verify this experimentally and show the variation of this convergence across corpora and tokenizations.

To show this, we train lexinvariant and standard LMs with both character-level vocabulary (128 ascii characters) and T5 default vocab (32k tokens) over the three datasets. For each model, we measure the perplexity of each token in each sequence w.r.t. context length, smoothed by moving average within each sequence, i.e. $P(x_i,...,x_{i+k}|x_1,...,x_i)^{\frac{1}{k}}$ for context length i. We set the moving average window $k=100$. We plot results over 100 sequences. As shown in figure 3, the perplexity gap between lexinvariant LM and standard LM gradually shrinks as the prefix becomes longer and longer, albeit much more slowly with a larger vocabulary. This makes intuitive sense since a larger vocabulary has more possibilities of permutations and requires many more prefix tokens to disambiguate. For the 32K vocabulary, the 512 context length will only allow the model to see a very small number of tokens,

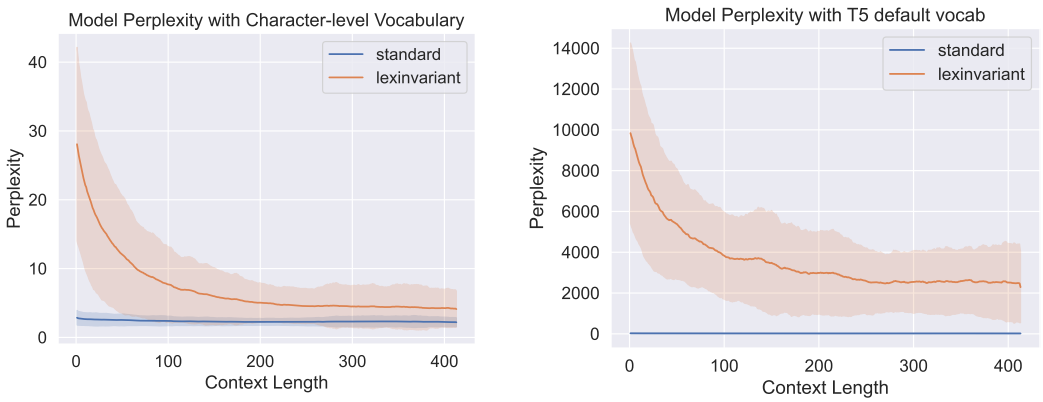


Figure 3: Perplexity over the Pile with character-level vocabulary (left) and T5 default vocab (right).

let alone to see tokens more than once. Nonetheless, the model still manages to show the trend of convergence, since even a small number of repetitions can form common patterns in grammar (such as the usage of spaces, punctuation, articles, etc). For the character-level vocabulary, the perplexity gap shrinks from 9X to less than 1X the average perplexity of the standard LM. With a context length of 511, the lexinvariant LM converges to perplexity 3.38, almost comparable to the perplexity of the standard LM of 2.00. Additionally, we observe that the gap shrinks significantly faster for models trained over Github than standard English text like Wiki-40B since code is more structured and it is easier to decipher the token permutation. We show the comparison across different datasets in Figure 7 in Appendix.

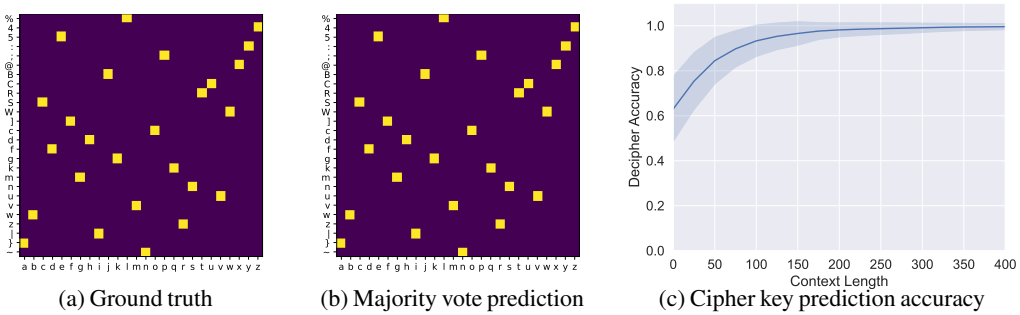
3.3 Recovering Substitution Ciphers

Here we show that lexinvariant LM is implicitly performing Bayesian in-context deciphering by testing its ability to recover cipher keys (e.g. Figure 4a) from character-level substitution ciphers, e.g. `uC; kvR5W 4mfzd @f| Svcgn fw;m uCRmu;;d ]%~} :fBn`. For the lexinvariant LM, this cipher text is perceived as the same as `the quick brown fox jumps over thirteen lazy dogs`, due to the lexinvariant property. It will then proceed to complete the cipher text with `%d: uC; @f|` with the same probability as it will complete the normal text with `and the fox`.

Because of this, we cannot directly read out the distribution of possible cipher keys $\mathcal{P}(\pi|x_1,...,x_{n-1})$ implicitly inferred by the lexinvariant LM. To do this, we train a small two-layer MLP probe on top of a frozen trained lexinvariant LM. For each training sequence, we first embed the input sequence with a randomly sampled token embedding E as described in section 2.3 and obtain the hidden activation of the final layer generated by the frozen lexinvariant LM. Then, we pass this activation through the two-layer MLP probe. Finally, instead of decoding the output activations to classification logits with the same E as in the lexinvariant LM, we instead use another learnable non-randomized token embedding matrix E' so that the probe can recover the deciphered token with stable token embeddings. Overall, we train the probe jointly with this embedding matrix E' to predict the current token. Effectively, we are training the probe to decipher the current token using the representation provided by the lexinvariant LM. We train the probe over the same corpus as the original lexinvariant LM for 10k steps. With this probe, we can directly visualize $\mathcal{P}(\pi^{-1}(x_n)|x_1,...,x_n)$ inferred by the lexinvariant LM, which is effectively one row in the permutation matrix representing π.

Now we can use this probe to explicitly recover the cipher key. An example ground truth cipher key that we want to recover is shown in Figure 4a. Note that although the substitution cipher is only among lowercase letters, the character-level lexinvariant model we use assumes that all permutations among the 128 characters are possible , making the deciphering even more challenging.

Concretely, we first input ciphertext through the frozen lexinvariant LM with the probe to produce a deciphered sequence. We then select a window of size 100 in the middle of the sequence and perform a majority vote over the corresponding deciphered tokens of each cipher token seen in this window. This essentially produces a predicted cipher key matrix for each window, and we can measure its precision against the ground truth. As shown in Figure 4c, such a cipher key prediction generally has increasingly higher precision as the window is selected later in the context, and it becomes near-perfect by the end of the sequence. Specifically, the cipher key matrix produced by the last window has an average precision of 99.6% over 1000 input sequences.



(a) Ground truth (b) Majority vote prediction (c) Cipher key prediction accuracy

Figure 4: (a) (b): Cipher key matrix, where the vertical axis shows the cipher characters and the horizontal axis shows the deciphered letters. The highlighted entries show the correspondences between cipher characters and the actual letters, e.g. `%` deciphers to `l`. (c): Cipher key prediction accuracy, averaged across 1000 input sequences. Context length denotes the start index of the window.